Lab 46 — Log Analysis with Regular Expressions

> **Course:** CompTIA Certified A+ Training (Core 1 and Core 2) (TGS-2024048317)
> **Topic 08:** Software Troubleshooting — Core 2, 23% of Core 2
> **Exam objective:** Filter and parse system logs with regular expressions to isolate the events that matter during troubleshooting (Core 2 objectives 3.1 and 4.8).

Goal

Logs contain thousands of lines and perhaps three that matter. You build the regular expressions that find those three, testing each pattern in RegexLab before applying it to real log data on the Killercoda playground.

What you'll produce

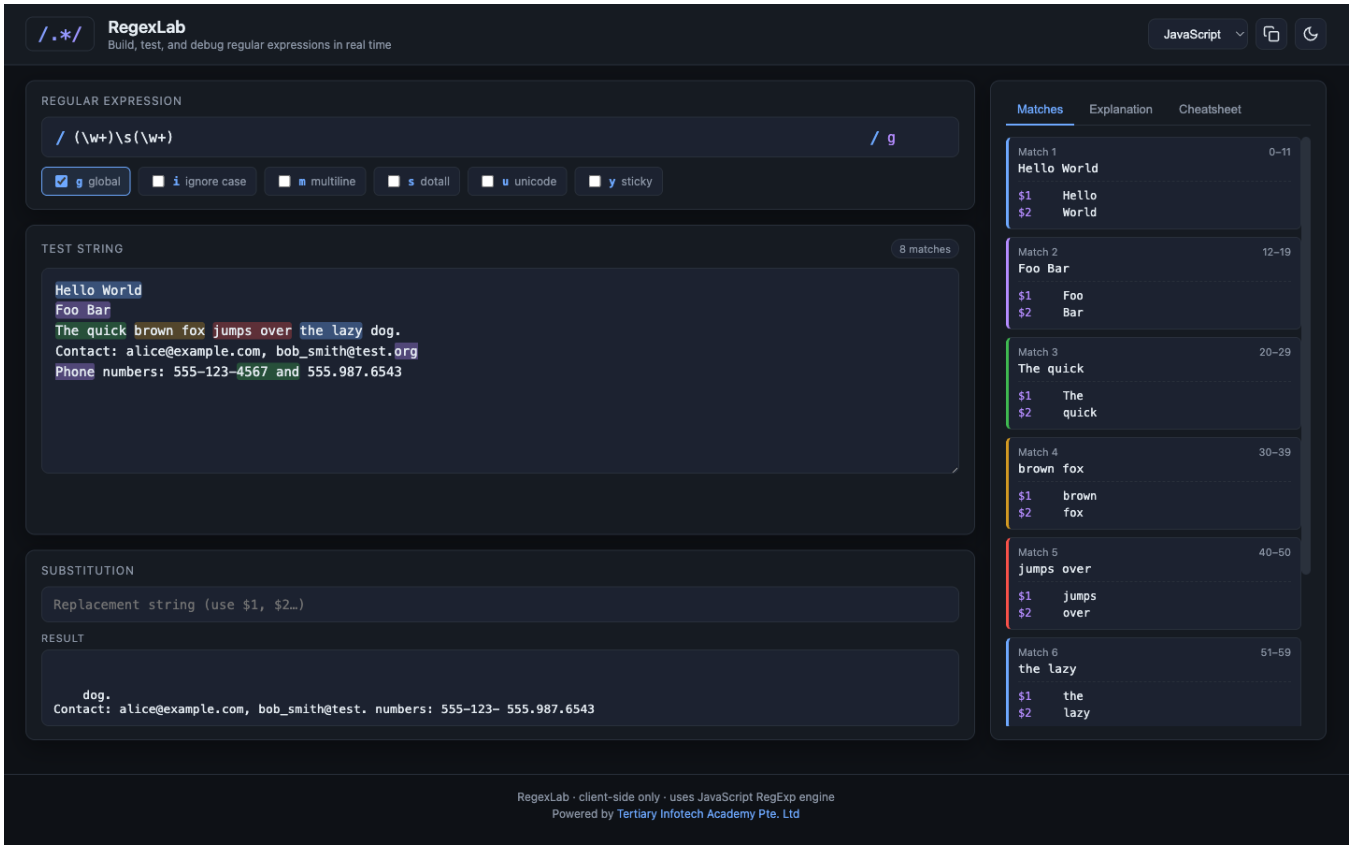
A tested regular expression library for log analysis, applied to real log files with the matches verified.

Tools and equipment

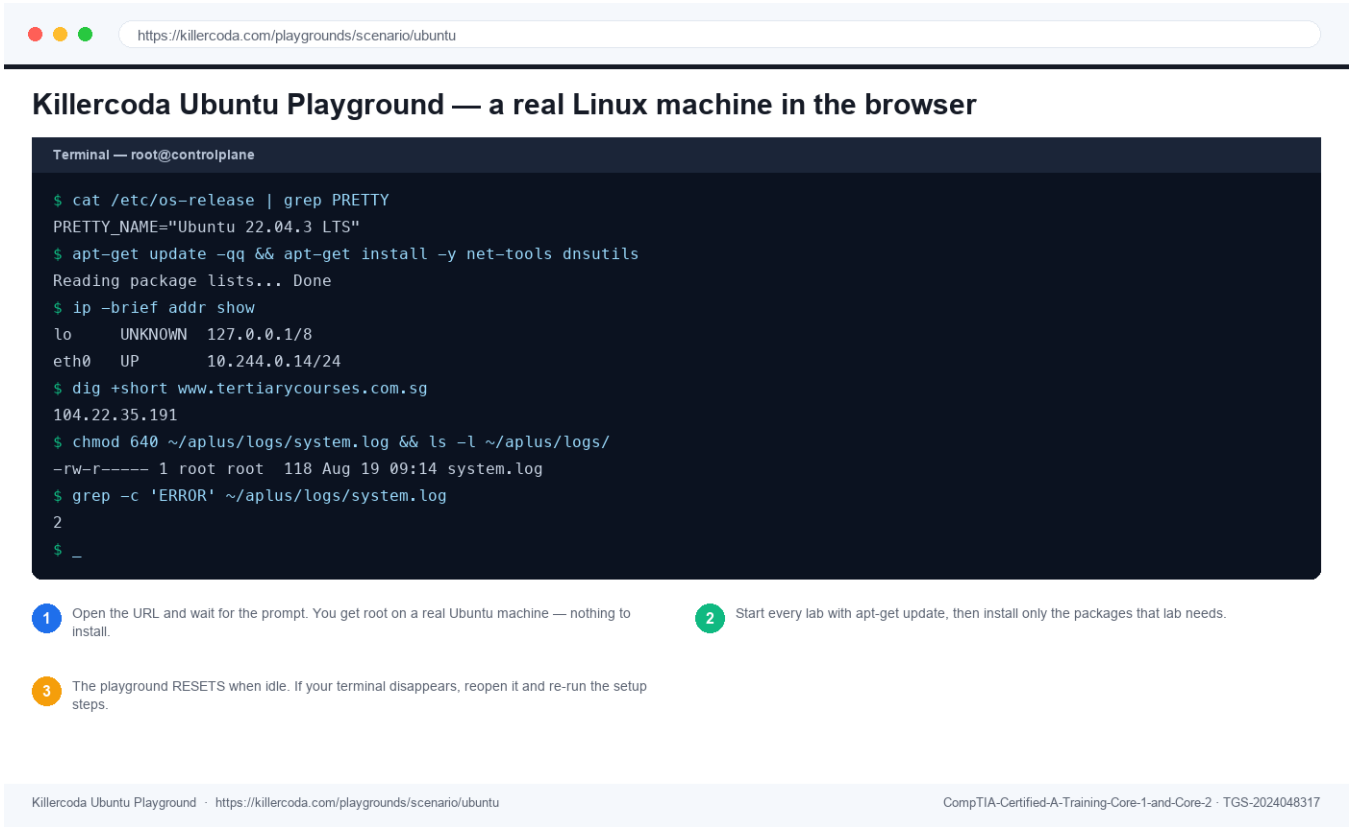
RegexLab (https://alfredang.github.io/regexgenerator/), Killercoda Ubuntu Playground, grep

Browser tools used in this lab

- **RegexLab** — https://alfredang.github.io/regexgenerator/
- **Killercoda Ubuntu Playground** — https://killercoda.com/playgrounds/scenario/ubuntu



RegexLab — the panels and fields this lab uses.



Killercoda Ubuntu Playground — the panels and fields this lab uses.

Safety

- Disconnect mains power and hold the power button for 15 seconds before working inside any machine.
- Wear an anti-static strap connected to an earthed point; hold boards and cards by their edges.
- **Never open a power supply unit or a CRT monitor** — both retain a lethal charge after being unplugged.
- Never puncture, compress or charge a swollen lithium battery; isolate it and follow hazardous disposal procedure.

Step-by-step

1. Open RegexLab at https://alfredang.github.io/regexgenerator/ and review the cheatsheet covering character classes, anchors, quantifiers and groups.
2. Record the core building blocks: \d matches a digit, \w a word character, \s whitespace, . any character, ^ start of line, $ end of line and \b a word boundary.
3. Build a pattern matching an IPv4 address and test it against sample text: \b\d{1,3}.\d{1,3}.\d{1,3}.\d{1,3}\b. Record the match count.
4. Build a pattern matching a timestamp in HH:MM:SS format and record the matches: \b\d{2}:\d{2}:\d{2}\b.
5. Build a pattern matching error severity keywords using alternation and the ignore-case flag: (ERROR|CRITICAL|FATAL|WARN).
6. Build a pattern matching an email address and test it against the sample text already loaded in the tool.
7. Build a pattern matching a MAC address in colon-separated form: ([0-9A-Fa-f]{2}:){5}[0-9A-Fa-f]{2}.
8. Experiment with the flags and record what each changes: g for all matches, i for case insensitivity, m for multiline anchors and s for dot matching newlines.
9. Open the Killercoda Ubuntu playground and create a realistic log file to apply your patterns to.

```
cat > /root/app.log << 'EOF'

2026-08-19 09:14:22 INFO  Service started on 192.168.1.10

2026-08-19 09:15:03 ERROR Connection refused from 10.0.0.55

2026-08-19 09:15:44 WARN  Disk usage 91% on /dev/sda1

2026-08-19 09:16:10 ERROR Auth failed for user admin from 203.0.113.9

2026-08-19 09:17:55 INFO  Backup completed

2026-08-19 09:18:31 CRITICAL Database unreachable at 10.0.0.80

EOF
```

```
cat /root/app.log
```

10. Apply the severity pattern with grep and confirm it returns exactly the error, warning and critical lines.

```
grep -inE '(ERROR|CRITICAL|WARN)' /root/app.log
```

11. Apply the IP address pattern to extract every address mentioned in the log, then sort them uniquely.

```
grep -oE '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' /root/app.log | sort -u
```

12. Combine patterns to answer a real support question — which IP addresses appear in error lines only — and record the answer.

```
grep -E '(ERROR|CRITICAL)' /root/app.log | grep -oE '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b'
| sort -u
```

Test it — verification

Every pattern is verified in RegexLab with its match count recorded; the severity grep returns exactly four lines from the log; the IP extraction returns four unique addresses; and the combined query correctly returns only the addresses from error and critical lines.

Troubleshooting this lab

Symptom	What to check
A command returns "command not found"	Re-run the `apt-get install` step at the start of the lab — the playground starts with a minimal package set.
The Killercoda terminal has reset	The playground times out when idle. Reopen it and re-run the setup commands from step 1.
A browser tool will not load	Check the URL against `labs/tools.md`. All four tools run entirely client-side and need no login.
Output differs from the guide	Record what you actually observed — your environment differs from the reference, and explaining the difference is part of the exercise.

Review questions

1. State the exam objective this lab maps to, in your own words.
2. Which single step in this lab would you perform first on a real support call, and why?
3. What evidence would you attach to a support ticket to show this work was completed correctly?
4. Name one thing that would make this procedure fail, and how you would recognise it.

Record your evidence

Complete worksheet.md as you work through this lab and keep it — the Practical Performance assessment mirrors these tasks.

← Labs index · Learner Guide · Course page